

BUT Informatique - R4.12

Automates et Langages

Bases de la Théorie des langages et des automates

J. Christian Attiogbé

Janvier 2025



J. Christian Attiogbé (Janvier 2025)

Automates et Langages

1 / 31

Contenu prévisionnel

Plan de la suite (adaptation selon contraintes)

- 1 Langages - Grammaires
- 2 Alphabet - Mot
- 3 Langages formels
- 4 Grammaires
- 5 Types de grammaires : Noam Chomsky



J. Christian Attiogbé (Janvier 2025)

Automates et Langages

2 / 31

Alphabet

Alphabet

Un alphabet (en théorie des langages) est un **ensemble fini non vide** de **symboles**.

Notation $\mathcal{A}$ l'alphabet ou Σ

Terminologie : **Symbole** = **lettre, caractère, signe**

Exemples d'alphabet:

- $\mathcal{A} = \{0, 1\}$
- $\mathcal{A} = \{a, z, i\}$
- $\mathcal{A} = \{-, ;, :, =,], [,), (\}$
- $\mathcal{A} = \{co, ri, ca, ci\}$
- $\mathcal{A} = \{end, begin, while, done, break, wait\}$

Mots

Mot

Un **mot** w est une **suite finie d'éléments définis sur un alphabet $\mathcal{A}$** donné. Un mot est donc défini sur $\mathcal{A} \times \mathcal{A} \times \mathcal{A} \times \dots$ (on verra la notation appropriée $\mathcal{A}^*$ dans la suite)

Le **mot vide** est autorisé et est noté ϵ

Exemples de mots

- 000001 011011001 1010101010 111 10001
- iza azi aziza izazi iizzzzaaaaaa zazizaza
- ;-) :=) :- ([:: (; :- (:-))) [-] ; -
- cocorico cacorici caricoco cacirica
- begin end end end begin
while begin end wait while begin
done break while end while while end end

Mots

Longueur d'un mot

La **longueur d'un mot** w est le nombre de symboles qui composent le mot ; c'est **un entier nul ou positif**.

On note $|w|$ la **longueur** de w , et $|\epsilon| = 0$

Ensemble de mots

$\mathcal{A}^*$ représente **les mots de longueur quelconque y compris le mot vide** (ens. des mots de longueur ≥ 0)

$$\mathcal{A}^* = \{w \mid \exists n \in \mathbb{N} \wedge w \in \mathcal{A}^n \quad (n \geq 0)\}$$

$\mathcal{A}^+$ représente **les mots constitués d'au moins un symbole** ; ce sont les mots de longueur supérieure ou égale à 1.

$$\mathcal{A}^+ = \{w \mid w \in \mathcal{A}^* \text{ et } |w| \geq 1\} \quad \mathcal{A}^* = \mathcal{A}^+ \cup \{\epsilon\}$$

Concaténation, préfixe, suffixe, facteur

Concaténation, préfixe, suffixe, facteur

Etant donné deux mots, w_1 et w_2 , on peut concaténer w_1 et w_2 pour former un mot w constitué de w_1 **suivi de** w_2 , et de longueur $|w| = |w_1| + |w_2|$.

Notation

L'opération de **concaténation** est souvent notée **.** comme $w_1.w_2$

Mais on peut l'omettre quand il n'y a pas d'ambigüité, comme w_1w_2

Exemples

La concaténation :

jour.bon donne le mot **jourbon**

bon.jour donne le mot **bonjour**

bonjour.ε donne **bonjour**

Propriétés de la concaténation

Propriétés

La **concaténation** peut se faire à droite ou à gauche d'un mot. Mais la concaténation **n'est pas commutative**. Elle **est associative**. ϵ est **un élément neutre** pour la concaténation.

On dit que $(\mathcal{A}, .)$ forme un *monoïde*.

On peut concaténer un même mot w autant de fois (en le précisant) qu'on veut ; par exemple, si $w = \text{til}$, alors $w^3 = (w)^3 = \text{tiltiltil}$

Exemples

jour.bon $\neq$ bon.jour

(bon.jour).bahia = bon.(jour.bahia)

Préfixe - suffixe de mots

Préfixe

On peut extraire les **préfixe** et **suffixe** d'un mot, en précisant leurs longueurs.

On peut décider si un mot est préfixe ou un suffixe d'un autre.

Préfixe

Un mot p est **préfixe** d'un mot w sur un alphabet $\mathcal{A}$ s'il existe un mot r tel que $p.r = w$

Suffixe

On définit le **suffixe** de façon similaire.

Les *chaînes de caractères* ou *string* en programmation sont des exemples triviaux de mots.

Langage formel

Un **langage** permet d'**utiliser** et **composer des mots** pour **exprimer** des idées, des intentions, des concepts, des réalités, presque tout. On distinguera la **formation/structuration des mots et des expressions** (la **syntaxe**), de leur **sens (sémantique)**.

Langage

Un langage $\mathcal{L}_A$, défini sur un alphabet $\mathcal{A}$, est un sous-ensemble de mots de $\mathcal{A}^*$:

$$\mathcal{L}_A \subseteq \mathcal{A}^*$$

$$\begin{aligned} 0010100111 &\in \{0,1\}^* \\ \{0111, 010100, 0101011\} &\subset \{0,1\}^* \end{aligned}$$

Opérations sur les langages

Les langages étant des ensembles de mots, on définit simplement plusieurs opérations ensemblistes sur les langages.

Union de langages

L'**union** de deux langages $\mathcal{L}_1$ défini sur un alphabet $\mathcal{A}_1$ et $\mathcal{L}_2$ défini sur $\mathcal{A}_2$ est le langage défini par l'ensemble des mots appartenant à $\mathcal{L}_1$ ou à $\mathcal{L}_2$:

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \vee w \in \mathcal{L}_2\}$$

Intersection de langages

L'**intersection** de deux langages $\mathcal{L}_1$ défini sur un alphabet $\mathcal{A}_1$ et $\mathcal{L}_2$ défini sur $\mathcal{A}_2$ est le langage défini par l'ensemble des mots appartenant à la fois à $\mathcal{L}_1$ et à $\mathcal{L}_2$:

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \wedge w \in \mathcal{L}_2\}$$

Opérations sur les langages (suite)

Différence de langages

$$\mathcal{L}_1 / \mathcal{L}_2 = \{w | w \in \mathcal{L}_1 \wedge w \notin \mathcal{L}_2\}$$

Produit de langages

$$\mathcal{L}_1 . \mathcal{L}_2 = \{w \mid \exists m \in \mathcal{L}_1 \wedge \exists v \in \mathcal{L}_2 \wedge w = m.v\}$$

Exemple

$$\{a, b\} . \{0, 1\} = \{a0, a1, b0, b1\}$$

Opérations sur les langages (suite)

Quotient de langages

$\mathcal{L}_2$ par $\mathcal{L}_1$

$$\mathcal{L}_1^{-1} . \mathcal{L}_2 = \{v \mid \exists m \in \mathcal{L}_1 \wedge m.v \in \mathcal{L}_2\}$$

Exemple

$$\{a, b\}^{-1} . \{a0, a1, b0, b1\} = \{0, 1\}$$

Opérations sur les langages (suite)

Puissance de langages

Puissance de $\mathcal{L}$: elle est définie inductivement

$$\mathcal{L}^0 = \{\epsilon\}$$

$$\mathcal{L}^{n>0} = \mathcal{L}.\mathcal{L}^{n-1}$$

Exemple

$$\mathcal{L} = \{0, 1\}$$

$$\mathcal{L}^2 = (\{0, 1\})^2 = \{00, 01, 10, 11\}$$

$$\mathcal{L}^3 = \{00, 01, 10, 11\}.\{0, 1\} = \{000, 010, 100, 110, 001, 011, 101, 111\}$$

Opérations sur les langages (suite)

Clôture/fermeture (de KLEENE) de langage

La clôture ou fermeture d'un langage est définie comme :

$$\mathcal{L}^* = \bigcup_{i \geq 0} (\mathcal{L}^i)$$

$$\mathcal{L}^+ = \mathcal{L}^* \setminus \{\epsilon\}$$

Opérations sur les langages (suite)

Constats

On voudrait bien former des mots spécifiques, et **faire des suites de mots spécifiques**, donc faire des **phrases** = avec une structure particulière.

Cela revient à se donner des **règles de formation des mots** et des langages.

Grammaires

Grammaire : introduction

Un **langage** peut être contraint par des **règles particulières pour structurer la suite de mots** de l'alphabet considéré.

Grammaire

Une grammaire est un ensemble de règles qui engendre un langage.

Structure - syntaxe

Toutes les phrases construites en respectant les règles de grammaire d'un langage **n'ont pas forcément de sens**.

On peut exprimer des phrases sans tête ni queue ; par exemple :

le berger gerbé a mergé chantant en !

Grammaire

Grammaire

Une grammaire (notée G) est un quadruplet $\langle T, N, S, R \rangle$ avec

- T est un alphabet, appelé le **vocabulaire terminal** ; ce sont les éléments qui vont apparaître dans le langage engendré.
- N est un ensemble de symboles, appelé le **vocabulaire non terminal**, qui n'apparaîtront pas dans les mots générés ; ils seront remplacés par des terminaux ; ils sont utilisés pour la génération.
- ...

Grammaire (suite)

Grammaire (suite)

Une grammaire (notée G) est un quadruplet $\langle T, N, S, R \rangle$ avec

- ...
- S est le **symbole de départ**, **Start** ou **axiome** de la génération du langage. S est élément de N . La génération des mots commencera par S en utilisant des règles pour produire des transformations de S et d'autres éléments de N qui sont introduits.
- R est un ensemble de règles dites **règles de production**.
Les règles de production ont la forme : $L \rightarrow R$ où
 $L \in (N \cup T)^+$ et $R \in (N \cup T)^*$
Les règles produisent, progressivement, la structure des mots du langage voulu.

Grammaire (suite)

On appelle aussi un symbole **non terminal**, une **catégorie syntaxique**.

Les **éléments de $R \in (N \cup T)^*$** sont appelés des **formes**.

Une forme peut être transformée en une autre en appliquant une règle de production.

Une **règle de la forme $L \rightarrow T^*$** est appelée **une règle terminale** (on ne pourra plus transformer la partie droite).

Exemple de grammaire

Exemple de grammaire : G_1

$$G_1 = \langle$$

$T =$	$\{x, ;, -, =, +, *, 2, 7\},$	% les terminaux
$N =$	$\{V, E, P, I, OP, SI\},$	% les non terminaux
$S =$	$P,$	% P est l'axiome
$R =$	$\{P \rightarrow I \ SI$ $SI \rightarrow ; \ I \ SI$ $SI \rightarrow \epsilon$ $I \rightarrow V = V \ OP \ E$ $V \rightarrow x$ $OP \rightarrow + \mid - \mid *$ $E \rightarrow 2 \mid 7 \mid V \ OP \ E$ $\}$	% R contient les règles

$$\rangle$$

Exemple de grammaire (suite)

Grammaire : G_2

```

 $G_2 = \langle$ 
   $T = \{a, b, z, f, v, i, o, n, t\},$     % les terminaux
   $N = \{P, M, I\},$                     % les non terminaux
   $S = P,$ 
   $R = \{P \rightarrow aMa$ 
       $M \rightarrow bIb \mid zIz \mid fIf \mid nIn \mid tIt \mid vIv$ 
       $I \rightarrow i \mid o$ 
       $\}$ 
 $\rangle$ 

```

Exemple de grammaire (suite)

Grammaire : SQL (un extrait)

```

query : values
      | WITH withItem [ , withItem ]* query
      | {
          select
          | selectWithoutFrom
          | query UNION [ ALL | DISTINCT ] query
          | query EXCEPT [ ALL | DISTINCT ] query
          | query MINUS [ ALL | DISTINCT ] query
          | query INTERSECT [ ALL | DISTINCT ] query
        }
      [ ORDER BY orderItem [ , orderItem ] * ]
      [ LIMIT { count | ALL } ]

```

Exemple de grammaire (suite)

Grammaire : SQL (suite)

```
select:    SELECT [ ALL | DISTINCT ]
          { * | projectItem [, projectItem ]* }
          FROM tableExpression
          [ WHERE booleanExpression ]
          [ GROUP BY [ ALL|DISTINCT ] { groupItem [, groupItem]* } ]
          [ HAVING booleanExpression ]
```

Dérivation

Une forme non vide (g) peut **être transformée par une grammaire** en une forme éventuellement vide d .

Dérivation simple

On dit qu'il y a **dérivation de g en d en une étape**, et on note cela $g \Rightarrow d$, lorsqu'il existe une règle de la grammaire G qui permet de transformer g en d (en tenant compte des préfixes ou suffixes éventuels de g et d).

Dérivation multiple

Par généralisation, on effectue **des dérivations en une ou plusieurs étapes** (éventuellement vides): $g \Rightarrow^+ d$ ou $g \Rightarrow^* d$.

Langage engendré par une grammaire

Langage engendré par une grammaire

Une grammaire engendre un langage : l'ensemble des mots obtenus en **partant de l'axiome** de la grammaire et en appliquant les règles de production, donc **des dérivations**, jusqu'à ne plus avoir de non terminaux dans les mots.

Le **langage engendré par une grammaire** $G = \langle T, N, S, R \rangle$ **est l'ensemble des mots (terminaux = dans T) obtenus par dérivations successives à partir de l'axiome de G .** On note ce langage $\mathcal{L}(G)$

$$\mathcal{L}(G) = \{w \mid w \in T^* \wedge S \Rightarrow^+ w\}$$

Langage engendré par une grammaire

Exemple 1

les **trois** (3) mots suivants sont engendrés par la grammaire G_1

$$\begin{aligned} x &= x-7; x = x+x+2; x = x+x-x-x-7 \\ x &= x-2; x = x-x-2; x = x+x-x-x^*7 \\ x &= x^*7; x = x^*x-2; x = x^*x^*x^*x^*7; x = x-2 \end{aligned}$$

Exemple 2

Les **huit** mots suivants font partie des mots engendrés par la grammaire G_2

aziza, abiba
aziza, anona, afifa
azozza, afofa, aviva

Langage engendré par une grammaire

Remarque

En **fonction de la structuration des règles** de production d'une grammaire, on a :

- **plus ou moins de restrictions** pour la construction des mots,
- et ainsi **différents types de grammaires**.

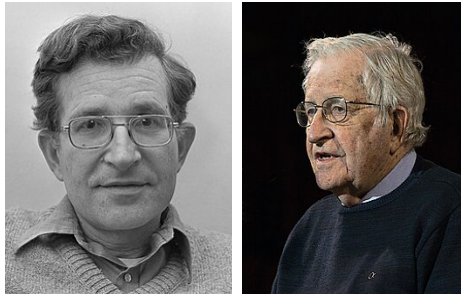


Figure: N. Chomsky (wikipedia)

Types de grammaires

Les restrictions imposées sur la forme des règles aboutissent à une **classification et une hiérarchisation des grammaires**.

Quatre (4) types de grammaire ont été définis en 1957 par N. CHOMSKY.

Type 0 - il n'y a aucune restriction sur la structure des règles

Les grammaires de ce type permettent d'engendrer tous les langages qui peuvent être reconnus en un temps fini par une machine ; ils sont dit **langages décidables**.

Types de grammaires

Les restrictions imposées sur la forme des règles aboutissent à une **classification et une hiérarchisation des grammaires**.

Quatre (4) types de grammaire ont été définis en 1957 par N. CHOMSKY.

Type 0 - il n'y a aucune restriction sur la structure des règles

Les grammaires de ce type permettent d'engendrer tous les langages qui peuvent être reconnus en un temps fini par une machine ; ils sont dit **langages décidables**.

Type 1 - *Context-dependent grammar*

Ces grammaires sont dites **sensibles au contexte ou contextuelles**.

Les règles de R sont de la forme : $cXc' \rightarrow cX'c'$.

La partie gauche d'une règle spécifie un contexte de dérivation pour X, avec $X \in N$ et $X' \in N$, $c \in (N \cup T)^+$ et $c' \in (N \cup T)^+$

Ces grammaires engendrent les **langages contextuels**, dépendant du contexte. Ils ne contiennent pas le mot vide ϵ

Types de grammaires (suite)

Type 2 - *Context-free grammar*

Grammaire hors-contexte ou **grammaire non-contextuelle**.

Les règles de R sont de la forme : $X \rightarrow X'$ avec $X' \in (N \cup T)^*$.

Elles ne spécifient pas de contexte, **le membre gauche est constitué d'un seul non terminal**.

Ces grammaires engendrent les **langages non contextuels**, indépendants du contexte, hors-contexte.

Types de grammaires (suite)

Type 3 - *Regular grammars* **Grammaires régulières.**

Lorsque les règles de G sont de la forme $X \rightarrow \alpha.X'$ où $\alpha \in T$ et $X, X' \in N$ la grammaire est dite **régulière à droite**.

Lorsque les règles de G sont de la forme $X \rightarrow X'.\alpha$ où $\alpha \in T$ la grammaire est dite **régulière à gauche**.

Les règles de la forme $X \rightarrow \alpha$ ou $X \rightarrow \epsilon$ sont admises.

Ces grammaires engendrent les **langages réguliers** (à gauche ou à droite). Langages réguliers = langages **rationnels** ou reconnaissables.

Conclusion

Hiérarchisation.

Les langages engendrés par ces types de grammaire sont ordonnés par une inclusion stricte. Notons $\mathcal{L}_{G_{type_i}}$ les langages engendrés par une grammaire de type i . On a alors :

$$\mathcal{L}_{G_{type3}} \subset \mathcal{L}_{G_{type2}} \subset \mathcal{L}_{G_{type1}} \subset \mathcal{L}_{G_{type0}}$$

Vous aurez remarqué l'importance et l'usage intensif des langages réguliers dans/pour les langages de programmation habituels et les langages de commande (Systèmes - Unix/Linux).

Les instructions (et leurs arguments) : `regex`, `find`, `grep`, `ls...`